Computational Simulation of Hydroelectric Dam Systems

ENGT 509

Final Project

By

Rushendar Reddy

Table of Contents

List of Figures

1. Introduction

Hydroelectric power remains one of the most widely used renewable energy sources due to its efficiency, reliability, and low environmental impact compared to fossil fuel-based systems. A hydroelectric dam converts the potential energy of stored water into electrical energy by controlling water flow through turbines. The performance of such systems depends on several physical and operational parameters, including water height, flow rate, structural dimensions, and mechanical efficiency. Accurate evaluation of these factors is essential to ensure both safe operation and optimal energy production (White, 2016).

The structure of a dam is primarily influenced by hydrostatic pressure, which increases linearly with depth and results in a nonlinear distribution of force across the dam surface. This force must be carefully analyzed to prevent structural failure, especially under varying water levels. At the same time, the energy output of the system is governed by the flow of water and the effective conversion of hydraulic energy into electrical power. These interdependent relationships create a need for systematic analysis using mathematical and computational techniques (Munson et al., 2013).

Traditional analytical methods provide closed-form solutions for simplified cases; however, real-world systems often involve nonlinear behavior, parameter variability, and uncertainty. As a result, computational methods play a critical role in modern engineering analysis. Numerical integration techniques, such as the trapezoidal rule, enable approximation of forces when analytical solutions are impractical. Finite difference methods allow the discretization of differential equations to model pressure variations. Sensitivity analysis using derivatives helps quantify how small changes in input parameters affect system behavior. Furthermore, optimization techniques assist in identifying operating conditions that maximize performance while satisfying safety constraints (Chapra & Canale, 2018).

In addition to deterministic analysis, uncertainty in real-world systems must be considered. Parameters such as water flow rate and efficiency may vary due to environmental and operational factors. Monte Carlo simulation provides a probabilistic framework to model these variations and evaluate their impact on system performance. This approach allows engineers to assess risk and reliability under uncertain conditions, which is essential for large-scale infrastructure systems (Fishman, 1996).

The objective of this project is to develop a computational model for analyzing hydroelectric dam behavior using both analytical and numerical methods. The model evaluates hydrostatic pressure, total force on the dam, power generation, and safety factor while incorporating advanced computational techniques such as numerical integration,

finite difference approximation, optimization, and stochastic simulation. The developed system provides an interactive platform to study engineering trade-offs between safety and efficiency, demonstrating the practical application of computational methods in real-world engineering problems.

2. Problem Statement

Hydroelectric dam systems operate under varying water levels and flow conditions, which directly influence hydrostatic pressure distribution and structural loading on the dam body (White, 2016). Inadequate evaluation of these forces can lead to unsafe operating conditions, especially since hydrostatic force increases nonlinearly with water height, making the system highly sensitive to small parameter changes (Munson et al., 2013).

Existing analytical approaches provide simplified solutions but do not fully capture parameter variability, uncertainty, and the need for optimization in real-world scenarios, thereby requiring the integration of computational methods such as numerical integration, finite difference approximation, and stochastic simulation (Chapra & Canale, 2018).

Therefore, a computational framework is required to model dam safety and power generation while incorporating deterministic and probabilistic methods to support efficient and reliable operation, aligning with the National Academy of Engineering's Grand Challenge of providing access to clean and sustainable energy (National Academy of Engineering, 2008).

3. Literature Review

Hydroelectric dam systems are governed by principles of fluid mechanics, where hydrostatic pressure increases linearly with depth and produces a nonlinear distribution of force on the dam structure (White, 2016). Analytical expressions derived from integration are commonly used to compute total force acting on the dam surface. These formulations provide a theoretical foundation for understanding structural loading but assume idealized conditions that do not fully represent real-world variability (Munson et al., 2013).

The course on applied computational methods emphasizes the use of numerical techniques when analytical solutions are limited or when system parameters vary. Numerical integration methods, such as the trapezoidal rule, allow approximation of hydrostatic forces by discretizing the depth domain into finite segments. Finite difference methods are introduced to approximate differential equations, enabling the modeling of pressure variation with respect to depth. These methods provide flexibility in handling complex or non-uniform systems (Chapra & Canale, 2018).

Derivative-based analysis is used to study sensitivity, which helps quantify how changes in water height affect the resulting force on the dam. This concept is essential in engineering systems where small variations in input parameters can lead to significant changes in output. Additionally, optimization techniques are applied to determine operating conditions that maximize power generation while maintaining structural safety constraints (Rao, 2009).

Real-world systems are subject to uncertainty due to environmental and operational factors. Monte Carlo simulation, introduced as a probabilistic computational method, is used to model variability in parameters such as water height, flow rate, and efficiency. This approach provides insight into system reliability and risk, which cannot be captured using deterministic methods alone (Fishman, 1996).

The integration of these computational methods into a single framework enables a more comprehensive analysis of hydroelectric dam performance. This project applies the concepts learned in the course to develop a model that combines analytical solutions with numerical and probabilistic methods, providing a practical understanding of how computational techniques support engineering decision-making.

4. Mathematical Modelling

The mathematical model was developed to calculate hydrostatic pressure, total water force, hydroelectric power, and dam safety factor. The model uses water height, dam width, flow rate, turbine efficiency, and dam type as the main input parameters. These values are entered through the MATLAB interface and used to update all calculations and graphs.

4.1. Input Parameters:

The simulation uses the following input variables:

- $H = water\ height$
- $W = dam\ width$
- $Q = water\ flow\ rate\ (\frac{m^3}{s})$
- $e = turbine\ efficiency$
- $\rho = 1000\ kg/m^3$ (density of water)
- $g = 9.81\ m/s^2$ (gravitational force)

These values are used because the project models water under standard engineering assumptions (White, 2016).

4.2 Hydrostatic Pressure Calculation:

Hydrostatic pressure depends on depth below the water surface. The pressure equation is,

$$\boldsymbol{P = \rho g h} \quad (1)$$

Where,

"h" is the depth below the water surface. At the bottom of the dam, the depth equals the full water height "H".

As water height increases, pressure also increases proportionally. And maximum pressure is at bottom of the dam.

4.3 Total Hydrostatic Force Using Integration:

Pressure is not constant across the dam height. It is zero at the water surface and maximum at the bottom. Therefore, total force is calculated by integrating pressure over the dam surface.

A small horizontal strip of dam surface has area,

$$dA = W \times dh$$

Suppose small force acting on this strip is,

$$dF = P \times dA$$

Substituting (P =ρ g h),

$$df = \rho g h \times W \times dh$$

The total force is obtained by integrating from the water surface to the bottom,

$$\boldsymbol{F = \int_0^H \rho g h\, W\, dh}$$

Since "ρ", "g", and "W" are constants,

$$F = \rho g W \int_0^H h\, dh$$

$$\boldsymbol{F = \frac{1}{2}(\rho g H^2 W)} \qquad (2)$$

Equation 2 is used to calculate analytical force.

Here force increases rapidly with height. Since force depends on "H^2".

4.4 Hydroelectric Power Calculation

The hydroelectric power generated by the system is calculated using,

$$\boldsymbol{Power(P) = \eta \rho g Q H} \qquad (3)$$

where "Q" is the flow rate and "η" is turbine efficiency.

Since efficiency (η), density (ρ), gravity (g), and height (H) remain constant, power depends directly on flow rate (Q). This means that power increases proportionally with flow

rate, resulting in a linear relationship. Therefore, the Power vs Flow Rate graph is a straight line.

4.5. Safety Factor Calculation

The safety factor compares the maximum dam capacity with the calculated water force:

$$\boldsymbol{Safety\ Factor(SF) = \frac{F_{max}}{F}} \quad (4)$$

where "F_{max}" is the assumed maximum force capacity of the selected dam type.

The safety condition is interpreted as:

- Safe: $SF \geq 2$
- Warning: $1 \leq SF \leq 2$
- Unsafe: $SF < 1$

4.6. Summary of Mathematical Models Used

In this project following mathematical equations are used,

- $Presuure\ (p) = \rho gh$
- $F = \frac{1}{2}(\rho g H^2 W)$
- $Power(P) = \eta \rho g Q H$
- $Safety\ Factor(SF) = \frac{F_{max}}{F}$

5. Computational Methods

This section explains the computational methods used in the project along with the mathematical calculations performed.

5.1 Numerical Integration Using Trapezoidal Rule

The total force acting on the dam is calculated using integration,

$$\boldsymbol{F = \int_0^H \rho g h W \, dh} \tag{5}$$

Where:

- ρ= density of water
- g = gravity
- h = depth
- W = width of dam

We know the **analytical solution** of this integral is,

$$F = \frac{1}{2}(\rho g H^2 W)$$

To approximate this using **numerical methods**, the trapezoidal rule is used:

$$F \approx \frac{\Delta h}{2}[f(h_0) + 2f(h_1) + 2f(h_2) + 2f(h_3) + \ \ldots\ldots\ldots\ldots + f(h_n)] \tag{6}$$

Here,

- $f(h) = \ \rho g h W$
- $\Delta h = step\ size$
- $h_0 = 0\ \ (surface)$
- $h_n = H\ (bottom)$

In MATLAB, this is done by using:

$$\boldsymbol{Force_{trap} = trapz(h, \rho * g * h * W)} \tag{7}$$

5.2. Derivative-Based Sensitivity Analysis

The force equation is:

$$F = \frac{1}{2}(\rho g H^2 W)$$

To find how force changes with height, we take derivative,

$$\frac{df}{dH} = \rho g H W$$

This shows how sensitive the force is to changes in water height.

In MATLAB, this is computed using,

$$\boldsymbol{\frac{dF}{dH} \approx gradient(F)} \quad (8)$$

5.3. Finite Difference Approximation

This method converts the differential equation into a discrete form for numerical computation.

Pressure increases with depth accordingly,

$$\frac{dp}{dh} = \rho g$$

Using finite difference method,

$$\boldsymbol{P(i) = P(i-1) + \rho\, g\, \Delta h} \quad (9)$$

here,

- Δh = step size
- P(0) = 0 at surface

Eq(9) is used to calculate pressure step by step.

5.4 Optimization Under Constraints

The goal is to maximize power:

$$Power = \eta \rho g Q H$$

Constraint,

$$Saftey\ Factor = \frac{F_{max}}{Force} \geq 1.5$$

Substituting force:

$$\frac{\boldsymbol{F_{max}}}{\left(\frac{\mathbf{1}}{\mathbf{2}}\right)\boldsymbol{\rho g H^2 W}} \geq \mathbf{1.5} \qquad (10)$$

The MATLAB code checks different values of H and selects the best value that,

- gives maximum power
- satisfies safety condition

5.5 Monte Carlo Simulation for Uncertainty

This approach follows a Monte Carlo simulation method, where multiple simulations are performed using randomly varied input values. Each simulation represents a possible operating condition, and the resulting safety factors are used to evaluate system reliability and calculate risk. In MATLAB UI that built for this project, we can choose how many random samples we want.

Random variations include following variables,

- $H_{random} = H + random\ variable$
- $Q_{random} = Q + random\ variable$
- $\eta_{random} = \eta + random\ variable$

For each simulation following calculation were done,

- $F = \frac{1}{2}(\rho g H_{random}^2 W)$
- $Saftey\ Factor = \frac{F_{max}}{Force}$
- $Power = \eta_{random} * \rho * g * Q_{random} * H_{random}$

Risk is calculated as,

$$\boldsymbol{Risk}(\%) = \frac{\boldsymbol{Number\ of\ unsafe\ cases}}{\boldsymbol{Tota\ Cases}} \times \mathbf{100} \tag{11}$$

This shows how system behaves under uncertainty.

5.6 Dimensionless Validation and Scaling

A dimensionless parameter is defined as,

$$\pi = \frac{Force}{\rho g H^2 W}$$

Substituting force,

$$F = \frac{1}{2}(\rho g H^2 W)$$

So,

$$\boldsymbol{\pi = 0.5} \tag{12}$$

The simulation checks whether the computed value of π is close to 0.5. This shows that the force calculation follows the expected physical relationship and that the implementation is consistent with theory.

5.7. Summary

This project uses:

- Integration to calculate force
- Trapezoidal rule for numerical approximation
- Derivatives for sensitivity analysis
- Finite difference for pressure calculation
- Optimization to find best operating conditions
- Monte Carlo simulation for uncertainty

6. Implementation

6.1. MATLAB Environment and Tools

The project is implemented using MATLAB due to its strong support for numerical computation, visualization, and graphical user interface development. MATLAB provides built-in functions that simplify the implementation of computational methods used in this project.

Key MATLAB functions used include:

- **trapz()** for numerical integration using the trapezoidal rule
- **gradient()** for computing numerical derivatives
- **linspace()** for generating evenly spaced data points
- **randn()** for generating random values in Monte Carlo simulation
- **plot() and histogram()** for visualization of results

6.2. User Interface

The system includes a graphical user interface (GUI) that allows users to interact with the model. The interface is designed using MATLAB UI components such as sliders, text fields, and panels.

The following input controls are provided as showed in Figure 1:

- Water height (H) slider
- Dam width (W) slider
- Flow rate (Q) slider
- Efficiency (η) slider
- Dam type selection (gravity, arch, buttress, earth-fill)
- Numerical resolution control
- Monte Carlo sample size control

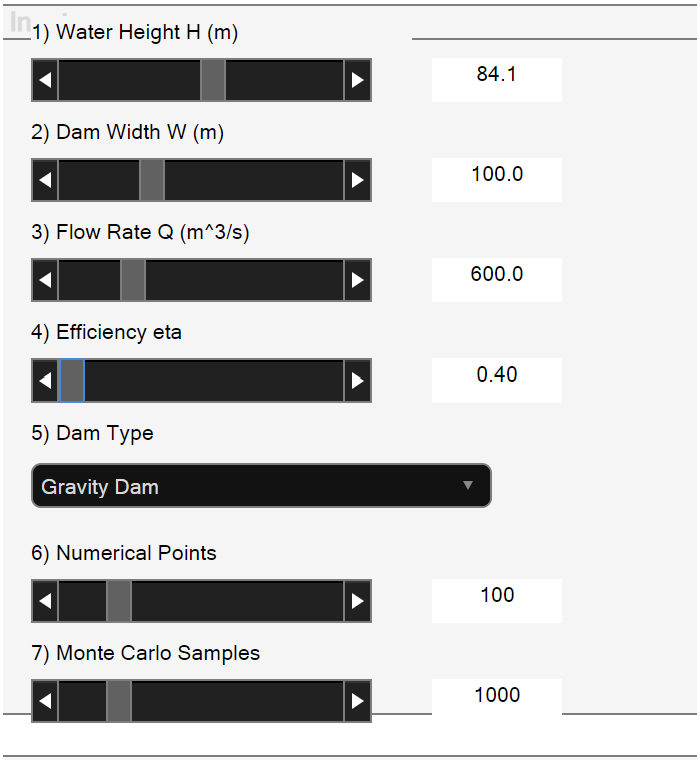

Figure 1. Input Variables for UI in Matlab

The output section displays as shown in Figure 2,

- Pressure at the bottom
- Total force on the dam
- Power generated
- Safety factor
- Risk percentage

In addition, multiple plots are included to visualize system behavior's shown in Figure 2,

- Force vs Height
- Power vs Flow Rate
- Sensitivity (Derivative) plot
- Monte Carlo distribution
- Finite difference pressure comparison

The interface updates automatically whenever an input parameter is changed.

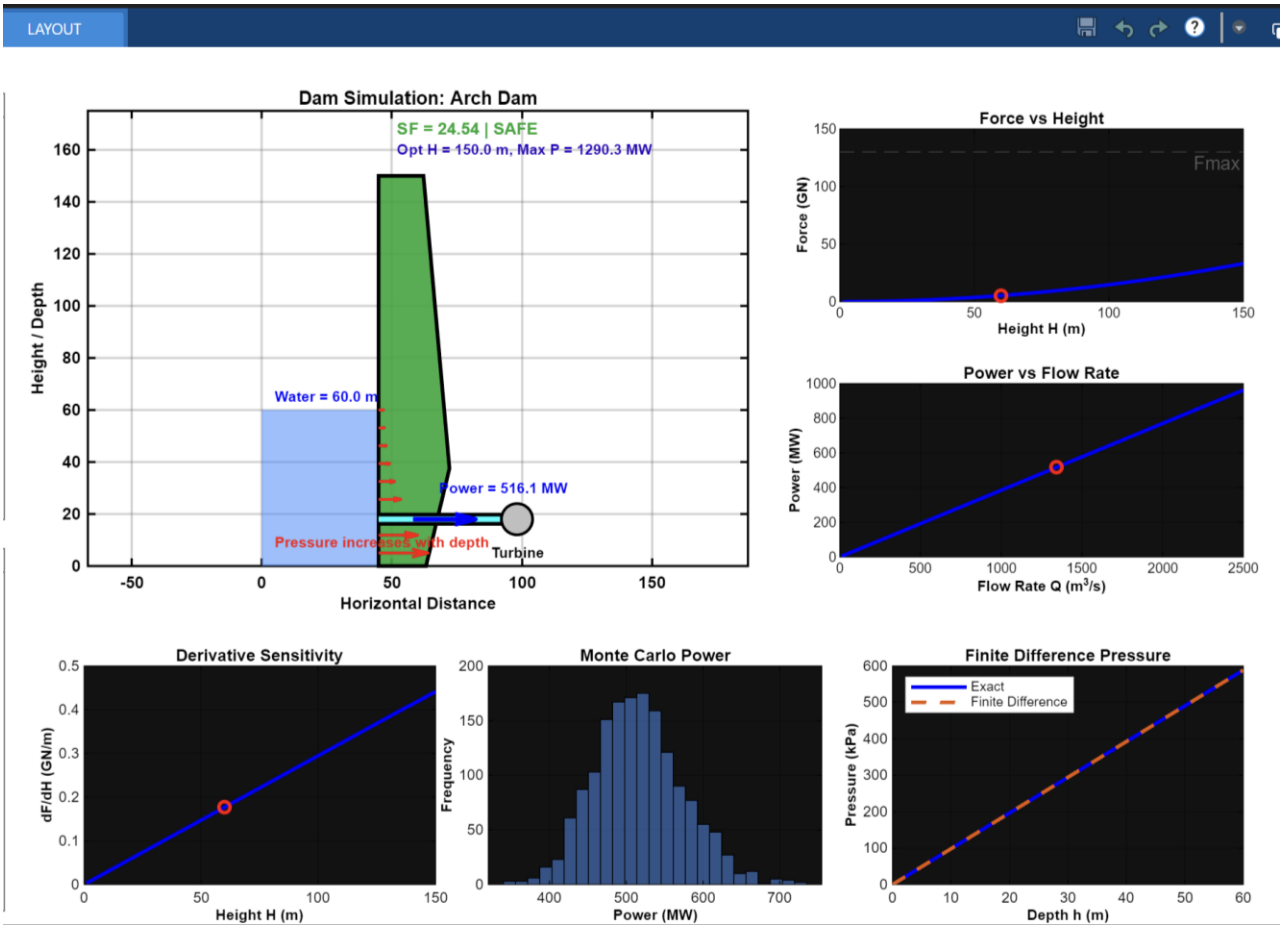

Figure 2. Output from UI in Matlab

7. Results and Discussion

This section presents the results obtained from the MATLAB simulation using the input parameters: water height H = 60 m, dam width W = 100 m, flow rate Q = 600 m^3/s, efficiency η = 0.85, and dam type selected as Gravity Dam.

(These values are selected as a representative case for the report, and the simulation allows different input values to be used for analyzing system behavior with different conditions.)

7.1. Force vs Height Analysis

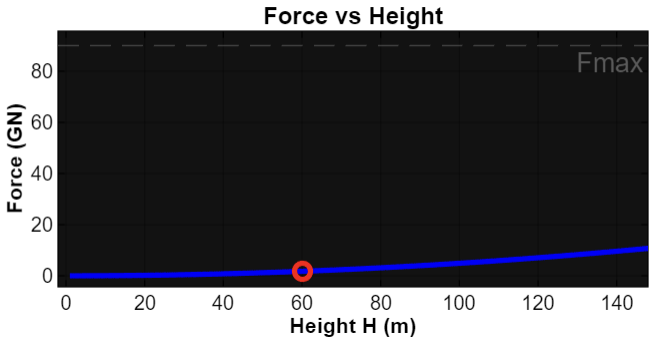

Figure 3. Force vs Height relationship

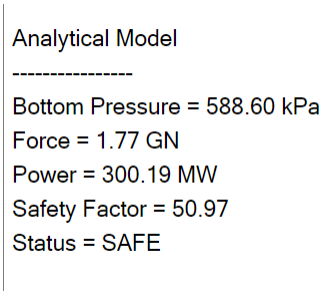

Figure 4. Analytical Values in UI from Matlab

The relationship between hydrostatic force and water height is shown in Figure 3. The curve demonstrates a nonlinear increase in force with respect to height.

At H = 60 m, the computed force acting on the dam is approximately 1.77 GN. The nonlinear trend confirms that force is proportional to the square of height, indicating that increases in water level leads to rapidly increasing structural loads as force increases.

7.2. Power vs Flow Rate Analysis

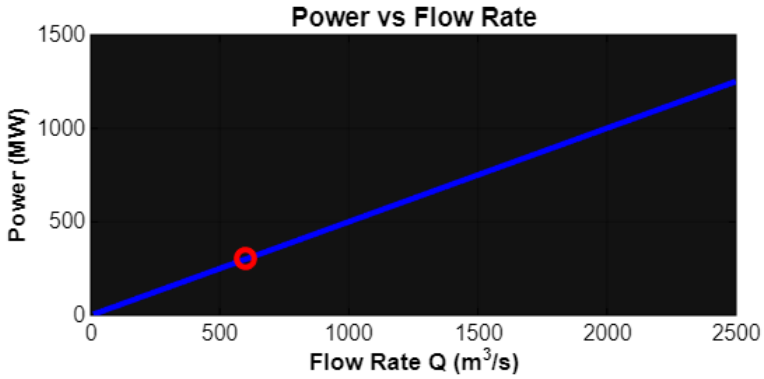

Figure 5. Power vs Flow Rate relationship

Figure 5 illustrates a linear relationship between power and flow rate. At Q = 600 m^3/s, the generated power is approximately 300.2 MW.

The linear trend confirms that power output increases proportionally with flow rate when other parameters remain constant. This matches the expected theoretical behavior of hydroelectric systems.

7.3. Sensitivity Analysis Using Derivatives

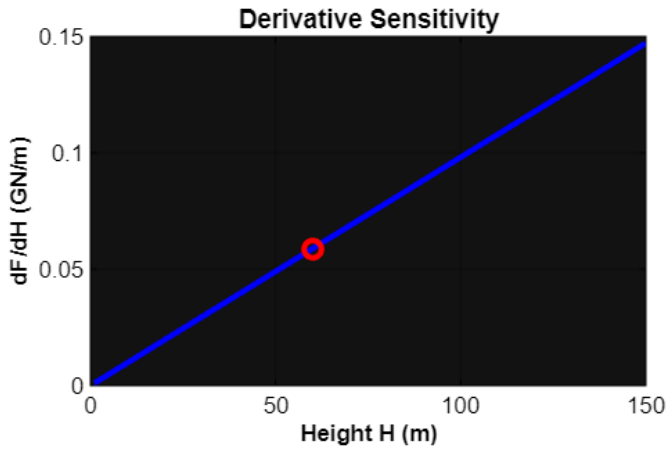

Figure 6. Sensitivity of force with respect to height

The derivative plot, Figure 6 shows how force changes with respect to water height. At H = 60 m, the rate of change of force is moderate, the system is less sensitive at lower heights and becomes more sensitive as height increases.

This suggests that the current operating condition is stable, and small variations in water height will not result in extreme changes in force.

7.4. Monte Carlo Simulation Results

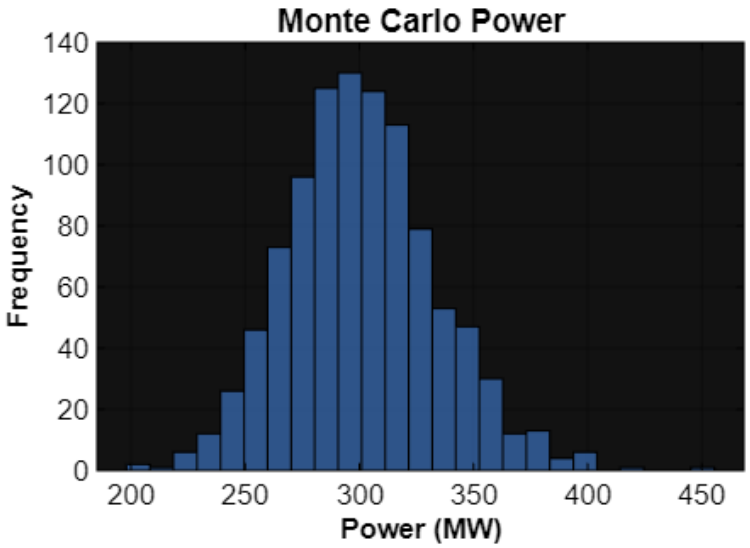

Figure 7. Monte Carlo distribution of power

The Monte Carlo simulation shows the distribution of power output under uncertain input conditions. The histogram indicates that the power values are centered around approximately 300 MW, with a relatively narrow spread.

This indicates low variability in system performance under uncertainty. The simulation also suggests negligible unsafe risk, confirming that the system operates safely even when input parameters fluctuate.

7.5. Finite Difference Validation

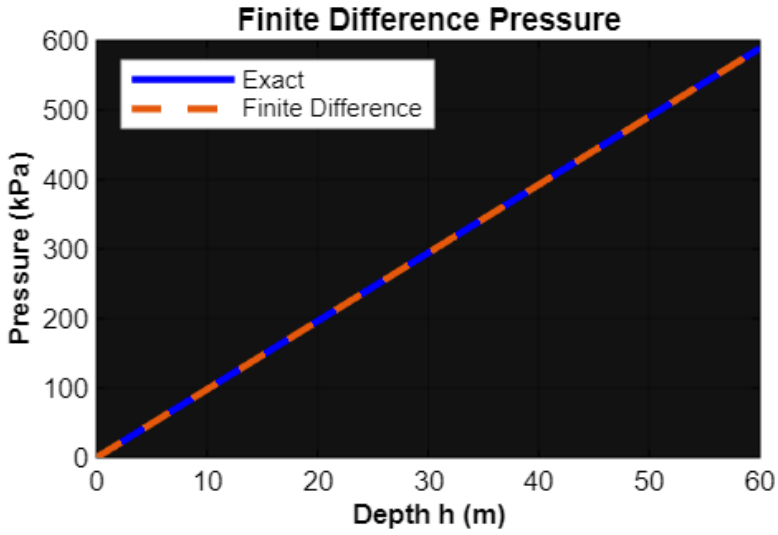

Figure 8. Comparison of analytical and finite difference pressure

The finite difference pressure profile closely matches the analytical solution, with both curves overlapping almost perfectly.

This occurs because the equation for pressure we used is linear. Since the pressure gradient is constant, the finite difference method reproduces the exact analytical solution. This result validates the accuracy of the numerical method implemented in the model.

7.6. Optimization Results

The optimization process determines the maximum achievable power while maintaining safe operating conditions. (*this calculated optimized values are different from user input values in our simulation, as these values are suggestions*)

```
Optimization
------------
Best H = 150.0 m, Max P = 750.5 MW
```

Figure 9. Calculated Optimization Output

The results indicate,

- Optimal Height = 150.0 m

- Maximum Power ≈ 750.5 MW

This demonstrates that increasing water height significantly improves power generation. However, such increases must be carefully managed to ensure structural safety.

7.7. Summary

The results highlight the fundamental trade-off between power generation and structural safety in hydroelectric dam systems. Increasing water height improves power output but also increases hydrostatic force which reduces the safety factor.

At the current operating condition (H = 60 m), the system produces approximately 300 MW while maintaining a high safety factor (SF ≈ 50.97), indicating a highly safe operating condition.

The sensitivity analysis shows that the system is less sensitive at lower heights, while the Monte Carlo simulation confirms robustness under uncertainty. The agreement between analytical and numerical methods further validates the computational model.

8. Conclusion

The project applies concepts from applied computational methods within a simulation environment in Matlab. These include integration for force calculation, numerical integration using the trapezoidal rule, derivative-based sensitivity analysis, finite difference methods for solving differential equations, constrained optimization and Monte Carlo simulation for uncertainty analysis. These methods are implemented together in the simulation to model the dam system under different input conditions.

9. Future Work

Future work will extend the current 2D simulation into a more realistic 3D model. Key aspects to consider include:

- 3D geometry of the dam (length, thickness, curvature)
- 3D pressure distribution and flow behavior
- Structural stress and material properties
- Use of advanced methods such as CFD and finite element analysis
- Improved 3D visualization of water flow and pressure
- Integration with real-world data or digital twin environments

These improvements will provide a more accurate and realistic analysis of dam performance.

10. References

White, F. M. (2016). *Fluid mechanics* (Eighth edition.). McGraw-Hill Education.

Munson, B. R., Gerhart, P. M., Gerhart, A. L., Hochstein, J. I., Young, D. F., & Okiishi, T. H. (2019). *Munson, Young, and Okiishi's Fundamentals of fluid mechanics* (8e ed.). Wiley.

Numerical methods for engineers: with personal computer applications: Steven C. Chapra and Richard P. Canale McGraw-Hill, March 1985, 400 pp., £43.95 ISBN 07 010664 9. (1986). *Engineering Analysis*, *3*(1), 62–62. https://doi.org/10.1016/0264-682X(86)90195-4

Fishman, G. (1996). Monte Carlo Concepts, Algorithms, and Applications. In *Monte Carlo*. Springer.

National Academy of Engineering. (2008). *Grand Challenges for Engineering*.

10. Appendix

Matlab code:

```
function AdvancedDamComputationalMethods_UI_v8
clc; close all;

%%

rho = 1000;      % kg/m^3
g   = 9.81;      % m/s^2

%% Figure
fig = figure('Name','Advanced Dam Computational Methods Simulator - Final Clean
Layout', ...
    'NumberTitle','off', ...
    'Position',[30 40 1350 720], ...
    'Color','w');

set(gcf,'Color','w');

%% ================================================
% LEFT SIDE PANELS
panelColor = [0.96 0.96 0.96];

inputPanel = uipanel('Parent',fig,'Title','Inputs', ...
    'FontSize',10,'FontWeight','bold', ...
    'BackgroundColor',panelColor,'ForegroundColor','k', ...
    'Position',[0.015 0.48 0.25 0.47]);

resultPanel = uipanel('Parent',fig,'Title','Scrollable Results', ...
    'FontSize',10,'FontWeight','bold', ...
    'BackgroundColor',panelColor,'ForegroundColor','k', ...
    'Position',[0.015 0.05 0.25 0.40]);

%% input controls
xLab = 0.05;
xSld = 0.05;
xVal = 0.72;

labelW = 0.62;
sliderW = 0.62;
valueW = 0.20;

hLabel = 0.055;
hSlider = 0.07;

y = 0.88;
dy = 0.125;

uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','1) Water Height H (m)', ...
```

```
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
heightSlider = uicontrol('Parent',inputPanel,'Style','slider','Units','normalized',
...
    'Min',5,'Max',150,'Value',60, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
heightText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','60','ForegroundColor','k','BackgroundColor','w');

y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','2) Dam Width W (m)', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
widthSlider = uicontrol('Parent',inputPanel,'Style','slider','Units','normalized',
...
    'Min',10,'Max',300,'Value',100, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
widthText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','100','ForegroundColor','k','BackgroundColor','w');

y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','3) Flow Rate Q (m^3/s)', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
flowSlider = uicontrol('Parent',inputPanel,'Style','slider','Units','normalized', ...
    'Min',10,'Max',2500,'Value',600, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
flowText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','600','ForegroundColor','k','BackgroundColor','w');

y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','4) Efficiency eta', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
etaSlider = uicontrol('Parent',inputPanel,'Style','slider','Units','normalized', ...
    'Min',0.4,'Max',0.95,'Value',0.85, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
etaText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','0.85','ForegroundColor','k','BackgroundColor','w');

y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','5) Dam Type', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
damMenu = uicontrol('Parent',inputPanel,'Style','popupmenu','Units','normalized', ...
    'String',{'Gravity Dam','Arch Dam','Buttress Dam','Earth-fill Dam'}, ...
    'Position',[xSld y-0.075 0.82 hSlider], ...
    'Callback',@updateSimulation);
```

```
y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','6) Numerical Points', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
resolutionSlider =
uicontrol('Parent',inputPanel,'Style','slider','Units','normalized', ...
    'Min',10,'Max',500,'Value',100, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
resolutionText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized',
...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','100','ForegroundColor','k','BackgroundColor','w');

y = y - dy;
uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xLab y labelW hLabel], ...
    'String','7) Monte Carlo Samples', ...
    'HorizontalAlignment','left','ForegroundColor','k','BackgroundColor',panelColor);
mcSlider = uicontrol('Parent',inputPanel,'Style','slider','Units','normalized', ...
    'Min',100,'Max',5000,'Value',1000, ...
    'Position',[xSld y-0.075 sliderW hSlider],'Callback',@updateSimulation);
mcText = uicontrol('Parent',inputPanel,'Style','text','Units','normalized', ...
    'Position',[xVal y-0.075 valueW hSlider], ...
    'String','1000','ForegroundColor','k','BackgroundColor','w');

%% Scrollable result box
resultBox = uicontrol('Parent',resultPanel,'Style','listbox','Units','normalized',
...
    'Position',[0.05 0.05 0.90 0.88], ...
    'FontSize',8.5, ...
    'ForegroundColor','k', ...
    'BackgroundColor','w', ...
    'String',{'Results appear here.'}, ...
    'Value',1);

%% ==============================
% PLOT LAYOUT
axVisual      = axes('Parent',fig,'Position',[0.31 0.43 0.38 0.50]);

axForce       = axes('Parent',fig,'Position',[0.74 0.72 0.23 0.19]);
axPower       = axes('Parent',fig,'Position',[0.74 0.44 0.23 0.19]);

axDerivative  = axes('Parent',fig,'Position',[0.31 0.08 0.20 0.24]);
axMonteCarlo  = axes('Parent',fig,'Position',[0.54 0.08 0.19 0.24]);
axFDPressure  = axes('Parent',fig,'Position',[0.77 0.08 0.20 0.24]);

updateSimulation();

%% ==========
% UPDATE FUNCTION

function updateSimulation(~,~)
```

```
H   = get(heightSlider,'Value');
W   = get(widthSlider,'Value');
Q   = get(flowSlider,'Value');
eta = get(etaSlider,'Value');
damType = get(damMenu,'Value');

nPoints = round(get(resolutionSlider,'Value'));
Nmc     = round(get(mcSlider,'Value'));

set(heightText,'String',sprintf('%.1f',H));
set(widthText,'String',sprintf('%.1f',W));
set(flowText,'String',sprintf('%.1f',Q));
set(etaText,'String',sprintf('%.2f',eta));
set(resolutionText,'String',sprintf('%d',nPoints));
set(mcText,'String',sprintf('%d',Nmc));

%% Dam strength assumptions
if damType == 1
    damName = 'Gravity Dam';
    Fmax = 9.0e10;
elseif damType == 2
    damName = 'Arch Dam';
    Fmax = 1.3e11;
elseif damType == 3
    damName = 'Buttress Dam';
    Fmax = 6.5e10;
else
    damName = 'Earth-fill Dam';
    Fmax = 4.5e10;
end

%% Analytical calculations
P_bottom = rho*g*H;
F_analytical = 0.5*rho*g*H^2*W;
Power = eta*rho*g*Q*H;
SF = Fmax/F_analytical;

%% Numerical integration
h_vals = linspace(0,H,nPoints);
pressure_vals = rho*g*h_vals;
force_density = pressure_vals*W;
F_trapz = trapz(h_vals,force_density);
integrationError = abs(F_analytical - F_trapz)/F_analytical*100;

%% Derivative
dF_dH_analytical = rho*g*H*W;
H_range = linspace(1,150,250);
F_range = 0.5*rho*g.*H_range.^2*W;
dF_dH_numerical = gradient(F_range,H_range);

%% Finite difference
dh = H/(nPoints-1);
P_fd = zeros(1,nPoints);
h_fd = linspace(0,H,nPoints);
```

```
for i = 2:nPoints
    P_fd(i) = P_fd(i-1) + rho*g*dh;
end

P_exact = rho*g*h_fd;
fdPressureError = max(abs(P_exact - P_fd))/max(P_exact)*100;

%% Optimization
H_candidates = linspace(5,150,500);
bestH = NaN;
bestPower = 0;

for i = 1:length(H_candidates)
    H_test = H_candidates(i);
    F_test = 0.5*rho*g*H_test^2*W;
    SF_test = Fmax/F_test;
    Power_test = eta*rho*g*Q*H_test;

    if SF_test >= 1.5 && Power_test > bestPower
        bestPower = Power_test;
        bestH = H_test;
    end
end

%% Monte Carlo
rng(1);
H_rand = H + randn(1,Nmc)*0.05*H;
Q_rand = Q + randn(1,Nmc)*0.08*Q;
eta_rand = eta + randn(1,Nmc)*0.04;

H_rand(H_rand < 1) = 1;
Q_rand(Q_rand < 1) = 1;
eta_rand(eta_rand < 0.1) = 0.1;
eta_rand(eta_rand > 1.0) = 1.0;

F_rand = 0.5*rho*g.*H_rand.^2*W;
SF_rand = Fmax./F_rand;
Power_rand = eta_rand.*rho.*g.*Q_rand.*H_rand;

probabilityUnsafe = sum(SF_rand < 1)/Nmc*100;
meanPowerMC = mean(Power_rand);
stdPowerMC = std(Power_rand);

%% Dimensionless validation
Pi_force = F_analytical/(rho*g*H^2*W);

%% Safety color
if SF >= 2
    damColor = [0.1 0.65 0.2];
    status = 'SAFE';
elseif SF >= 1
    damColor = [1.0 0.75 0.1];
    status = 'WARNING';
else
    damColor = [0.9 0.1 0.1];
```

```
        status = 'UNSAFE';
    end

    %% Main visual
    axes(axVisual);
    cla(axVisual);
    styleWhiteAxis(axVisual);
    hold(axVisual,'on');

    maxH = 150;
    xlim(axVisual,[-10 125]);
    ylim(axVisual,[0 maxH+25]);
    axis(axVisual,'equal');
    grid(axVisual,'on');
    box(axVisual,'on');

    xlabel(axVisual,'Horizontal
Distance','Color','k','FontWeight','bold','FontSize',9);
    ylabel(axVisual,'Height / Depth','Color','k','FontWeight','bold','FontSize',9);
    title(axVisual,['Dam Simulation: ',
damName],'Color','k','FontWeight','bold','FontSize',10);

    patch(axVisual,[0 120 120 0],[0 0 -8 -8],[0.55 0.42 0.25], ...
        'EdgeColor','none');

    patch(axVisual,[0 45 45 0],[0 0 H H],[0.25 0.55 1.0], ...
        'FaceAlpha',0.55,'EdgeColor','none');

    text(axVisual,5,H+5,sprintf('Water = %.1f m',H), ...
        'FontSize',8,'FontWeight','bold','Color','b');

    if damType == 1
        damX = [45 72 58 45];
        damY = [0 0 maxH maxH];
    elseif damType == 2
        damX = [45 63 72 62 45];
        damY = [0 0 maxH*0.25 maxH maxH];
    elseif damType == 3
        damX = [45 60 60 45];
        damY = [0 0 maxH maxH];
    else
        damX = [38 88 62 45];
        damY = [0 0 maxH maxH];
    end

    patch(axVisual,damX,damY,damColor, ...
        'FaceAlpha',0.9,'EdgeColor','k','LineWidth',2);

    if damType == 3
        for yy = 20:25:125
            plot(axVisual,[60 82],[yy yy-18],'k','LineWidth',2);
        end
    end

    arrowDepths = linspace(5,H,9);
```

```
for k = 1:length(arrowDepths)
    yArrow = arrowDepths(k);
    depthFromSurface = H - yArrow;
    arrowLength = 2 + 20*(depthFromSurface/H)^2;
    quiver(axVisual,45,yArrow,arrowLength,0,0, ...
        'Color','r','LineWidth',1.5,'MaxHeadSize',2);
end

text(axVisual,5,9,'Pressure increases with depth', ...
    'FontSize',8,'FontWeight','bold','Color','r');

pipeY = 18;
plot(axVisual,[45 95],[pipeY pipeY],'k','LineWidth',7);
plot(axVisual,[45 95],[pipeY pipeY],'c','LineWidth',3);

rectangle(axVisual,'Position',[92 pipeY-6 12 12], ...
    'Curvature',[1 1], ...
    'FaceColor',[0.75 0.75 0.75], ...
    'EdgeColor','k','LineWidth',1.5);

text(axVisual,88,pipeY-13,'Turbine', ...
    'FontSize',8,'FontWeight','bold','Color','k');

quiver(axVisual,58,pipeY,24,0,0, ...
    'Color','b','LineWidth',2.5,'MaxHeadSize',2);

text(axVisual,68,pipeY+12,sprintf('Power = %.1f MW',Power/1e6), ...
    'FontSize',8,'FontWeight','bold','Color','b');

text(axVisual,52,168,sprintf('SF = %.2f | %s',SF,status), ...
    'FontSize',9,'FontWeight','bold','Color',damColor);

if ~isnan(bestH)
    text(axVisual,52,160,sprintf('Opt H = %.1f m, Max P = %.1f MW', ...
        bestH,bestPower/1e6), ...
        'FontSize',8,'FontWeight','bold','Color',[0.15 0 0.75]);
else
    text(axVisual,52,160,'No safe height found for SF >= 1.5', ...
        'FontSize',8,'FontWeight','bold','Color','r');
end

hold(axVisual,'off');

%% Force plot
axes(axForce);
cla(axForce);
styleWhiteAxis(axForce);

plot(axForce,H_range,F_range/1e9,'b','LineWidth',2.0);
hold(axForce,'on');
plot(axForce,H,F_analytical/1e9,'ro','MarkerSize',6,'LineWidth',2.0);
yline(axForce,Fmax/1e9,'--','Fmax','Color',[0.35 0.35 0.35], ...
    'LabelHorizontalAlignment','right','LabelVerticalAlignment','bottom');
hold(axForce,'off');
```

```
    xlabel(axForce,'Height H (m)','Color','k','FontWeight','bold','FontSize',8);
    ylabel(axForce,'Force (GN)','Color','k','FontWeight','bold','FontSize',8);
    title(axForce,'Force vs Height','Color','k','FontWeight','bold','FontSize',9);
    set(gca, 'XColor','k', 'YColor','k')
    grid(axForce,'on');
    box(axForce,'on');

    %% Power plot
    axes(axPower);
    cla(axPower);
    styleWhiteAxis(axPower);

    Q_range = linspace(10,2500,250);
    P_range = eta*rho*g.*Q_range*H;

    plot(axPower,Q_range,P_range/1e6,'b','LineWidth',2.0);
    hold(axPower,'on');
    plot(axPower,Q,Power/1e6,'ro','MarkerSize',6,'LineWidth',2.0);
    hold(axPower,'off');

    xlabel(axPower,'Flow Rate Q
(m^3/s)','Color','k','FontWeight','bold','FontSize',8);
    ylabel(axPower,'Power (MW)','Color','k','FontWeight','bold','FontSize',8);
    title(axPower,'Power vs Flow Rate','Color','k','FontWeight','bold','FontSize',9);
    set(gca, 'XColor','k', 'YColor','k')
    grid(axPower,'on');
    box(axPower,'on');

    %% Derivative plot
    axes(axDerivative);
    cla(axDerivative);
    styleWhiteAxis(axDerivative);

    plot(axDerivative,H_range,dF_dH_numerical/1e9,'b','LineWidth',2.0);
    hold(axDerivative,'on');
    plot(axDerivative,H,dF_dH_analytical/1e9,'ro','MarkerSize',6,'LineWidth',2.0);
    hold(axDerivative,'off');

    xlabel(axDerivative,'Height H (m)','Color','k','FontWeight','bold','FontSize',8);
    ylabel(axDerivative,'dF/dH (GN/m)','Color','k','FontWeight','bold','FontSize',8);
    title(axDerivative,'Derivative
Sensitivity','Color','k','FontWeight','bold','FontSize',9);
    set(gca, 'XColor','k', 'YColor','k')
    grid(axDerivative,'on');
    box(axDerivative,'on');

    %% Monte Carlo
    axes(axMonteCarlo);
    cla(axMonteCarlo);
    styleWhiteAxis(axMonteCarlo);

    histogram(axMonteCarlo,Power_rand/1e6,25, ...
        'FaceColor',[0.25 0.5 0.85], ...
        'EdgeColor','k');
    xlabel(axMonteCarlo,'Power (MW)','Color','k','FontWeight','bold','FontSize',8);
```

```
    ylabel(axMonteCarlo,'Frequency','Color','k','FontWeight','bold','FontSize',8);
    title(axMonteCarlo,'Monte Carlo
Power','Color','k','FontWeight','bold','FontSize',9);
    set(gca, 'XColor','k', 'YColor','k')
    grid(axMonteCarlo,'on');
    box(axMonteCarlo,'on');

    %% Finite difference
    axes(axFDPressure);
    cla(axFDPressure);
    styleWhiteAxis(axFDPressure);

    plot(axFDPressure,h_fd,P_exact/1000,'b','LineWidth',2.0);
    hold(axFDPressure,'on');
    plot(axFDPressure,h_fd,P_fd/1000,'--','Color',[0.9 0.35 0.05],'LineWidth',2.0);
    hold(axFDPressure,'off');

    xlabel(axFDPressure,'Depth h (m)','Color','k','FontWeight','bold','FontSize',8);
    ylabel(axFDPressure,'Pressure
(kPa)','Color','k','FontWeight','bold','FontSize',8);
    title(axFDPressure,'Finite Difference
Pressure','Color','k','FontWeight','bold','FontSize',9);
    legend(axFDPressure,{'Exact','Finite Difference'}, ...
        'Location','northwest','TextColor','k','Color','w');
    set(gca, 'XColor','k', 'YColor','k')
    grid(axFDPressure,'on');
    box(axFDPressure,'on');

    %% Scrollable results
    if isnan(bestH)
        optText = 'No safe optimum found';
    else
        optText = sprintf('Best H = %.1f m, Max P = %.1f MW',bestH,bestPower/1e6);
    end

    resultLines = {
        'Advanced Dam Analysis'
        '---------------'
        sprintf('Dam: %s',damName)
        sprintf('H = %.2f m',H)
        sprintf('W = %.2f m',W)
        sprintf('Q = %.2f m^3/s',Q)
        sprintf('eta = %.2f',eta)
        ''
        'Analytical Model'
        '----------------'
        sprintf('Bottom Pressure = %.2f kPa',P_bottom/1000)
        sprintf('Force = %.2f GN',F_analytical/1e9)
        sprintf('Power = %.2f MW',Power/1e6)
        sprintf('Safety Factor = %.2f',SF)
        sprintf('Status = %s',status)
        ''
        'Computational Methods'
        '-----------------'
        sprintf('Trapz Force = %.2f GN',F_trapz/1e9)
```

```
        sprintf('Trapz Error = %.5f %%',integrationError)
        sprintf('dF/dH = %.2f GN/m',dF_dH_analytical/1e9)
        sprintf('FD Error = %.5f %%',fdPressureError)
        sprintf('Pi Check = %.3f',Pi_force)
        ''
        'Optimization'
        '---------'
        optText
        ''
        'Monte Carlo'
        '--------'
        sprintf('Mean Power = %.2f MW',meanPowerMC/1e6)
        sprintf('Std Dev = %.2f MW',stdPowerMC/1e6)
        sprintf('Unsafe Risk = %.2f %%',probabilityUnsafe)
        ''
        'Why These Methods Are Used'
        '--------------------'
        'Trapz: approximates total water force.'
        'Derivative: shows sensitivity to height.'
        'Finite difference: solves pressure variation.'
        'Optimization: finds safest high-power design.'
        'Monte Carlo: tests uncertainty in operation.'
        'Pi check: validates force scaling.'
        };

    set(resultBox,'String',resultLines,'Value',1);
end

%% ===================================
% HELPER FUNCTION

    function styleWhiteAxis(ax)

    set(ax, ...
        'Color','w', ...              % white background
        'XColor','k', ...             % X-axis + ticks BLACK
        'YColor','k', ...             % Y-axis + ticks BLACK
        'GridColor',[0.3 0.3 0.3], ...% darker grid
        'GridAlpha',0.25, ...
        'LineWidth',1.2, ...
        'FontSize',9, ...
        'FontWeight','bold');         % makes numbers darker
    ax.XAxis.Color = [0 0 0];
    ax.YAxis.Color = [0 0 0];
end

end
```